1. What is Normalization in Databases?

Normalization is a **database design technique** used to organize data in tables so that:

- **Data redundancy (duplicate data) is minimized**
- **Data integrity is improved**
- **Update, Insert, Delete anomalies are avoided**
- The database becomes **efficient and easier to maintain**

Normalization is mainly applied during **database design** and is based on a set of rules called **Normal Forms (NF)**.

Common normal forms:

1. **First Normal Form (1NF)**
2. **Second Normal Form (2NF)**
3. **Third Normal Form (3NF)**
4. **Boyce–Codd Normal Form (BCNF)** (advanced)

For most real-world systems, **3NF is usually sufficient**.

Example Scenario (Education System)

Suppose we store student course information in **one single table**.

Unnormalized Table (Before Normalization)

StudentID	StudentName	CourseID	CourseName	Instructor	InstructorPhone
1	Ravi	C101	SQL	Kumar	999111
1	Ravi	C102	C#	Suresh	999222
2	Anil	C101	SQL	Kumar	999111
3	Priya	C103	Python	Ramesh	999333

Problems

1. **Data Redundancy**

    - CourseName repeated
    - Instructor repeated
2. **Update Anomaly**
    - If Kumar's phone changes → update many rows.
3. **Insert Anomaly**
    - Cannot insert a course without a student.
4. **Delete Anomaly**
    - If Ravi leaves → course information may be lost.

1️⃣ First Normal Form (1NF)

Rule

A table is in **1NF if:**

- Each column contains **atomic (single) values**
- No **repeating groups**
- Each record can be **uniquely identified (Primary Key)**

Example of violation

StudentID	StudentName	Courses
1	Ravi	SQL, C#, Python

Here **Courses contains multiple values**.

Convert to 1NF

StudentID	StudentName	Course
1	Ravi	SQL
1	Ravi	C#
1	Ravi	Python

Now each column contains **single values**.

Table Structure

```
CREATE TABLE StudentCourses
(
   StudentID INT,
   StudentName VARCHAR(50),
   CourseName VARCHAR(50)
);
```

But redundancy still exists.

2️⃣ Second Normal Form (2NF)

Rule

A table is in **2NF if:**

1. It is already in **1NF**
2. **No Partial Dependency**

What is Partial Dependency?

When a column depends on **only part of a composite primary key**.

Example Table

StudentID	CourseID	StudentName	CourseName
1	C101	Ravi	SQL
1	C102	Ravi	C#
2	C101	Anil	SQL

Primary Key = **(StudentID, CourseID)**

Problem:

- **StudentName depends only on StudentID**
- **CourseName depends only on CourseID**

This violates **2NF**.

Convert to 2NF

Split the table into **three tables**.

Students Table

StudentID	StudentName
1	Ravi
2	Anil

```
CREATE TABLE Students
(
   StudentID INT PRIMARY KEY,
   StudentName VARCHAR(50)
);
```

Courses Table

CourseID	CourseName
C101	SQL
C102	C#

```
CREATE TABLE Courses
(
    CourseID VARCHAR(10) PRIMARY KEY,
    CourseName VARCHAR(50)
);
```

StudentCourse (Mapping Table)

StudentID	CourseID
1	C101
1	C102
2	C101

```
CREATE TABLE StudentCourse
(
    StudentID INT,
    CourseID VARCHAR(10),
    PRIMARY KEY(StudentID, CourseID)
);
```

Now:

- No partial dependency
- Data duplication reduced

3️⃣ Third Normal Form (3NF)

Rule

A table is in **3NF if:**

1. It is in **2NF**
2. **No Transitive Dependency**

What is Transitive Dependency?

When a column depends on **another non-key column**.

Example

CourseID	CourseName	Instructor	InstructorPhone
C101	SQL	Kumar	999111
C102	C#	Suresh	999222

Problem:

- **InstructorPhone depends on Instructor**
- Instructor depends on CourseID
- So **InstructorPhone indirectly depends on CourseID**

This is **Transitive Dependency**.

Convert to 3NF

Split into two tables.

Courses Table

CourseID	CourseName	InstructorID
C101	SQL	I1
C102	C#	I2

```
CREATE TABLE Courses
(
    CourseID VARCHAR(10) PRIMARY KEY,
    CourseName VARCHAR(50),
    InstructorID INT
);
```

Instructor Table

InstructorID	InstructorName	Phone
I1	Kumar	999111
I2	Suresh	999222

```
CREATE TABLE Instructors
(
    InstructorID INT PRIMARY KEY,
    InstructorName VARCHAR(50),
    Phone VARCHAR(20)
);
```

Now:

- No transitive dependency
- Clean separation of data

Final Normalized Structure (3NF)

Students

Students

StudentID (PK)

StudentName

Courses

Courses

CourseID (PK)

CourseName

InstructorID (FK)

Instructors

Instructors

InstructorID (PK)

InstructorName

Phone

StudentCourse

StudentCourse

StudentID (FK)

CourseID (FK)

Advantages of Normalization

1. **Eliminates Data Redundancy**
2. **Improves Data Consistency**
3. **Prevents Update Anomalies**
4. **Better Data Integrity**
5. **Efficient Storage**